Early Detection of Silent Training Failures in Neural Networks

Sohan Bhat
sohanrambhat@gmail.com

November 2026

Abstract

Neural networks can fail during training without raising an error. The run finishes and returns a weaker model that is easy to mistake for a hard problem. LHD (Layer Health Diagnostics) records four signals for every layer after each epoch and compares each one with the range it stays within during healthy runs of the same network. When a signal stays out of range for three epochs, LHD names the layer and a likely cause and stops the run. I tested LHD on 885 training runs of three small networks with frozen layers and dead units planted in known layers. LHD caught 87% of 470 broken runs, fired 4.6 epochs before the accuracy curve on the runs both caught, and flagged 3 of 150 healthy runs.

1 Introduction

A neural network can break during training without any error. The run finishes, the loss goes down, and the model is simply worse than it should be. Errors in deep learning code rarely raise an exception and usually show up only as lower accuracy (Chwilczyński et al., 2024). Because nothing crashes, a failure like this is easy to read as a hard problem, and the compute spent on the run is wasted.

I ran into this while training SignNet, a small CNN for the German Traffic Sign Recognition Benchmark (GTSRB) (Stallkamp et al., 2012). It stopped improving at 76% accuracy, and I spent weeks tuning it. The cause was one convolutional layer that I had left out of the optimizer, so its weights never trained. Once I fixed that, accuracy rose to 87%. The evidence was in the weights the whole time, since that layer never changed, but nothing in a normal training loop looks at it.

LHD (Layer Health Diagnostics) is a monitor that reads this evidence. After every epoch, it records four signals for each layer and compares them with the range they stay within during healthy runs of the same network. When one signal stays out of range for three epochs in a row, LHD stops the run and reports the layer, the signal, and a likely cause, such as a frozen layer or dead units.

I tested LHD on 885 training runs of a fully connected network, a CNN, and a small transformer, with frozen layers and dead units planted in known layers. LHD caught most broken runs, usually at the third epoch and often several epochs before the accuracy curve showed anything. It also caught more of the faults that did real damage. For frozen layers, a simple check for weights that never change did just as well and sooner. The clearest gain is on dead units. Healthy networks lose some units on their own, and the share differs a lot between networks, so the fixed thresholds that existing tools use either miss these faults or flag healthy runs. A healthy range measured for each layer caught more of them than any fixed threshold I tested, with only a few false alarms.

2 Related Work

Several tools already check training signals automatically. SageMaker Debugger has built-in rules such as DeadRelu, VanishingGradient, and UnchangedTensor (Amazon Web Services, n.d.a), and a rule can stop

the training job when it fires (Amazon Web Services, n.d.b). DeepDiagnosis checks a model after every epoch for eight symptoms, reports the layer, and suggests a fix, which is close to what LHD reports (Wardat et al., 2022). AutoTrainer checks gradients, weights, and the training curves for five problems and retrains the model with built-in fixes (Zhang et al., 2021). The wandb-callbacks package adds a warning for dead ReLU units (Groeger, n.d.).

All four compare each signal with a threshold chosen in advance. DeadRelu fires when more than 50% of a layer's units are inactive, DeepDiagnosis counts a layer as having dead nodes above 70%, and wandb-callbacks warns at 80% by default. The share of units that die in healthy training differs between networks and between layers, so one threshold can miss faults in one network and flag healthy runs in another. DeepDiagnosis reported dead nodes in 21 of the 35 CIFAR-10 models from the AutoTrainer benchmark that had no known bugs (Wardat et al., 2022). LHD instead learns a range for each layer, signal, and epoch from healthy runs of the same network.

Other work is further from LHD. Karpathy's recipe lists manual habits for catching silent failures (Karpathy, 2019), and wandb shows gradient and weight histograms that a person still has to read (Biewald, 2020). ART adds validation checks to a training pipeline at development time (Chwilczyński et al., 2024). UMLAUT checks a program against expert heuristics and shows readable error messages (Schoop et al., 2021), and DeepLocalize traces numerical errors back to the layer that caused them (Wardat et al., 2021). A study of model repair found that a random search did as well as or better than AutoTrainer (Kim et al., 2023).

3 Method

At the end of every epoch, LHD records four signals for each layer. Let W_e be the layer's weights and bias after epoch e. The weight change $\|W_e - W_{e-1}\|/\|W_{e-1}\|$ shows how far the weights moved, so a frozen layer reads exactly 0. The gradient norm is the mean of $\|\partial L/\partial W\|$ over the epoch's batches, and the weight norm is $\|W_e\|$. The dead unit fraction is the share of units whose ReLU output is 0 for every input in a fixed probe batch of 256 validation examples.

The healthy range comes only from calibration runs, which are trained with no fault from different seeds. For layer ℓ, signal s, and epoch e, let $a_{\ell se}$ and $b_{\ell se}$ be the lowest and highest calibration values. A value x is out of range when

$$x < a_{\ell se}(1-m) \quad \text{or} \quad x > b_{\ell se}(1+m) + f, \tag{1}$$

with margin $m = 0.5$ and floor $f = 0.05$. The floor keeps a signal that is 0 in every healthy run from flagging one unusual unit. A value that is not a finite number is always out of range.

LHD raises an alarm when one signal of one layer stays out of range for $p = 3$ epochs in a row. It names a likely cause from the pattern of that layer's four signals (Table 1), stops the run, and reports the layer and the cause. Figure 1 shows LHD stopping a CNN run with a frozen layer. The settings m, f, and p were fixed in commit `e83806f` before any training run started.

4 Experimental Setup

I trained three networks (Table 2). The fully connected network classifies Fashion-MNIST, the CNN classifies GTSRB, and the transformer predicts the next character of Tiny Shakespeare. Every run used Adam with a cosine schedule on one CPU thread. A broken run has one fault planted in one layer at the start of training. A frozen layer has its weights put back after every optimizer step, which is the same failure as a layer left out of the optimizer. A dead unit fault switches off 10%, 25%, and 50% of a layer's units in separate runs, so their ReLU output is 0 for every input. Frozen layers went into every layer and dead units

Table 1: How LHD names a cause. The rules are checked from top to bottom on the layer that raised the alarm.

When the alarmed layer shows	LHD names
a signal that is not a finite number	diverged
more dead units than its range allows	dead units
weight change below its range while the gradient is inside or above	frozen layer
weight change and gradient both below their ranges	vanishing gradient
weight norm above its range	runaway weights
any other pattern	unrecognized

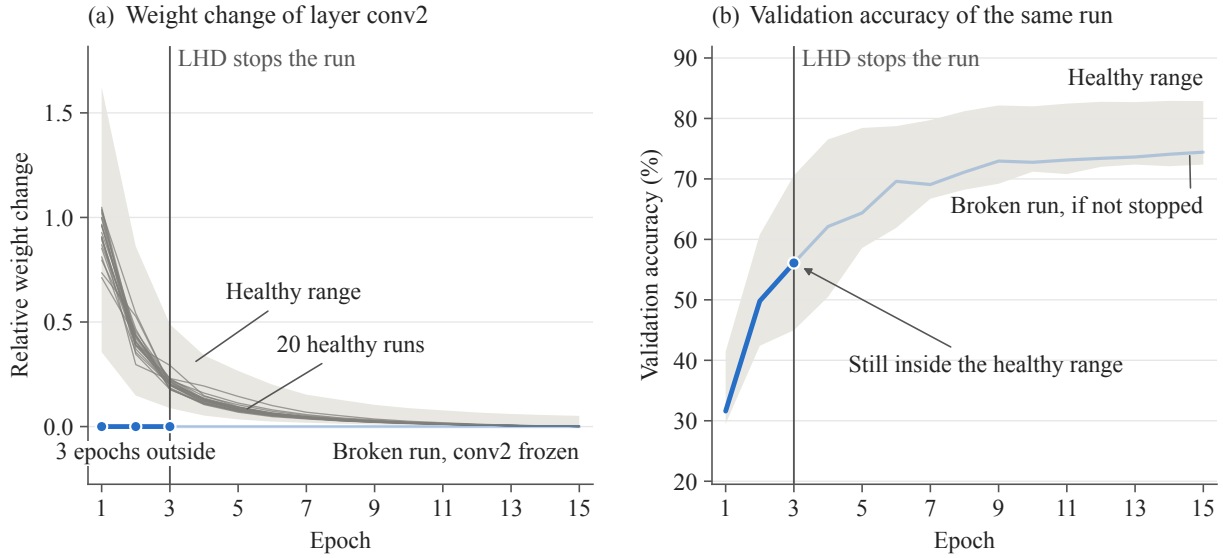


Figure 1: One CNN run with conv2 frozen. (a) The layer's weight change stays at 0, below the range of the 20 calibration runs, and LHD stops the run at epoch 3. (b) The run's validation accuracy is still inside the healthy range when LHD stops it.

into every layer followed by a ReLU. Calibration runs set the healthy ranges, and fresh healthy runs with other seeds measure false alarms. The run plan in commit `b6bfb6a` listed every run before training started.

The accuracy curve fires when validation accuracy stays below the lowest calibration value for p epochs in a row, with no margin. The zero update check fires when a layer's weights do not move at all in an epoch. The dead unit rules fire when a ReLU layer has at least 25%, 50% (SageMaker Debugger), 70% (DeepDiagnosis), or 80% (wandb-callbacks) of its units dead. Each rule must hold for p epochs, like LHD, and a broken run counts only when the rule names the planted layer.

5 Results

The per epoch curves, per layer results, signal ablation, settings checks, cost, extra faults, and per condition statistics are in the extended results file in the repository (`paper/short/lhd_extended.pdf`).

5.1 LHD and the accuracy curve

LHD caught 411 of the 470 broken runs (87%, 95% Wilson interval 84 to 90%), and the accuracy curve caught 166 (Table 3). LHD's median alarm came at epoch 3, the earliest possible for a fault planted at epoch

Table 2: The three networks and their runs. Calibration runs set the ranges, fresh healthy runs measure false alarms, and the results use the runs with a frozen layer or dead units. The other faults are in the extended results.

Network	Dataset	Parameters	Epochs	Calibration	Fresh healthy	Frozen or dead	Other faults
Fully connected	Fashion-MNIST	242,762	20	20	50	130	55
Transformer	Tiny Shakespeare	112,577	15	20	50	170	85
CNN	GTSRB	41,659	15	20	50	170	65
Total				60	150	470	205

Table 3: LHD next to the accuracy curve on the planted frozen layers and dead units. The median epoch is over the runs each detector caught. Bold marks the better of the two where they differ.

	Fully connected		Transformer		CNN	
	LHD	Accuracy	LHD	Accuracy	LHD	Accuracy
Broken runs caught	**84%**	26%	**100%**	48%	**78%**	29%
Median epoch of the alarm	**3**	17	**3**	5	**3**	7
Healthy runs flagged	1 of 50	1 of 50	**0 of 50**	6 of 50	2 of 50	**0 of 50**

1 when $p = 3$. On the 162 runs that both detectors caught, LHD fired 4.6 epochs earlier on average. The accuracy curve was earlier in 1 of those runs, and 26 were ties. The accuracy curve never noticed 61% of the runs LHD caught. Of all broken runs, 66% ended with a final accuracy inside the healthy range, so a person watching accuracy would have seen nothing wrong.

When LHD caught a run, it named the planted layer 98.1% of the time and the planted cause 96.8% of the time. Part of that is built in, since the planted faults produce the patterns the rules look for.

5.2 Faults that cost accuracy

A broken run and the fresh healthy run with the same seed start from the same weights and see the same batches, so they differ only by the fault. The healthy twin's final accuracy shows what the fault cost. Of the 105 broken runs whose fault cost 2 or more points of final accuracy, LHD caught 94 and the accuracy curve caught 60. A frozen CNN layer cost the most, 11.1 points on average, close to the 11 points that the untrained layer cost SignNet. Half of a CNN layer's units dead cost 3.1 points, and no fault in the other two networks cost more than 0.8 points on average.

5.3 Dead units and fixed thresholds

For frozen layers, the zero update check did better than LHD. It caught 200 of 200 frozen layers, each at the planted layer, and flagged no healthy runs. When one epoch is enough, it fires at epoch 1, 2 epochs before LHD. A frozen layer's weight change is exactly 0, so this check needs no healthy range at all.

Dead units are harder, because healthy networks lose units on their own and the amount varies widely. In calibration runs of the CNN, between 36% and 70% of the dense layer's units were dead, while no transformer calibration run had a single dead unit in its ReLU layers. A fixed threshold that suits one network will miss faults or flag healthy runs in another (Figure 2). Every rule here was held to the same $p = 3$ epochs and scored only at the planted layer. LHD caught 78% of dead unit faults with 3 false alarms in 150 healthy runs. The 25% rule caught 71% but flagged 51 healthy runs. The 50% rule, the SageMaker Debugger default, caught 40% and flagged 22. The 70% and 80% rules of DeepDiagnosis and wandb-callbacks flagged no healthy runs but caught only 4% and 0%. A rule that fires on any dead unit caught 74%

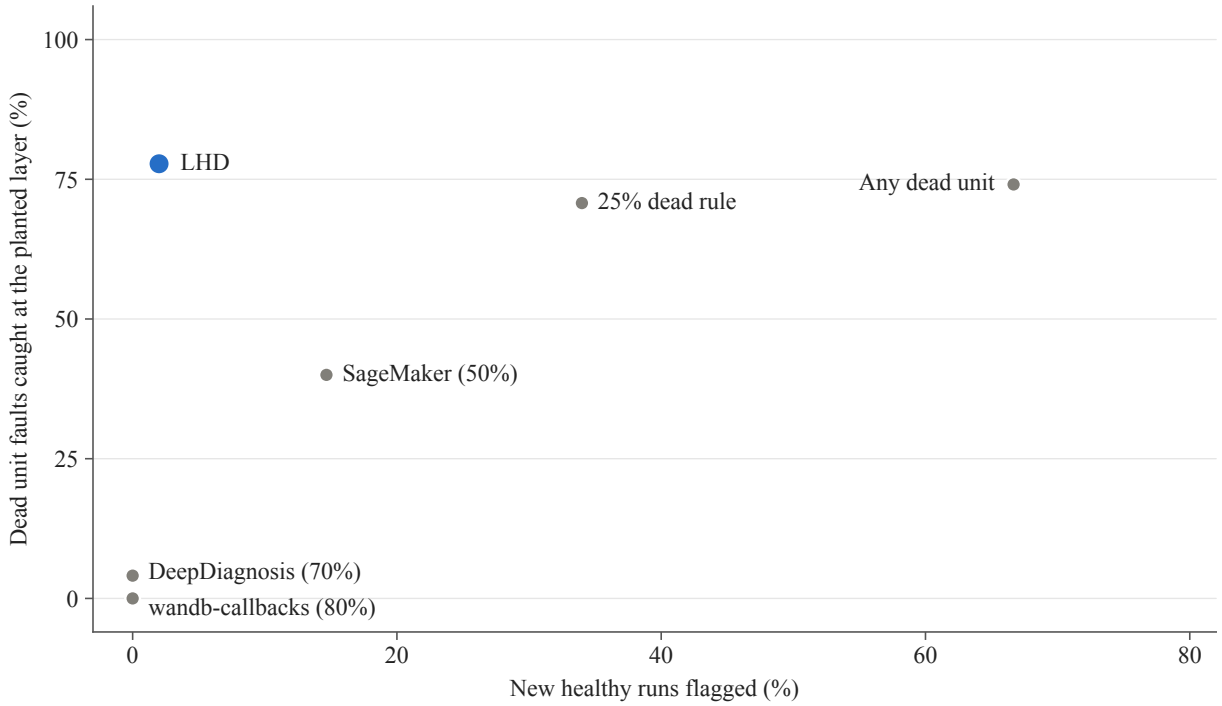


Figure 2: Dead unit faults caught at the planted layer against fresh healthy runs flagged, for LHD and the fixed dead unit rules, all held to three epochs and pooled over the three networks. Detectors toward the upper left catch more faults while flagging fewer healthy runs.

while flagging 100. A range measured for each layer of each network was the only detector that caught most dead unit faults without flagging many healthy runs.

5.4 False alarms

LHD flagged 3 of 150 fresh healthy runs (2.0%, 95% interval 0.7 to 5.7%), against 7 for the accuracy curve. All of them came from the dead unit fraction. Two were CNN runs in which one of the 16 filters of conv2 died, at epochs 3 and 4. No calibration run had a dead filter in conv2, so the top of that layer's range was the floor alone, 5.0%, and one dead filter is 6.25%. The third was in fc2 of the fully connected network at epoch 16. That layer had fewer dead units than any calibration run, 1.6% against a lowest healthy value of 3.9%, which put it under the lower edge of 2.0%. LHD called this pattern unrecognized. Too few dead units points to no fault, so checking the dead unit fraction against only the top of its range would remove this alarm. An earlier draft of this work, at commit `d3f5275`, reported the same 3 alarms when 110 healthy runs had finished, and the 40 runs trained after it added none.

6 Limitations

For frozen layers, the zero update check catches the fault two epochs sooner than LHD, so LHD's advantage is on dead units. The faults were planted to show up in the signals LHD measures, so they are not real bugs, and the rates above describe how well LHD separates these faults from normal variation. Calibration needs about 10 to 20 healthy runs of the same network, and it assumes those runs are healthy. For a model trained only once, that costs more than LHD can save.

LHD misses dead units in layers where healthy runs already lose many units. The CNN's dense layer is the clearest case, where LHD caught 1 of 10 runs even with half of its units dead. Many planted faults were also mild, since 223 of 470 cost less than half a point of accuracy. Only the frozen layer and dead unit rules were tested in the main study, and the divergence rule never fired in any run. The floor of 0.05 is too low for small layers, because one dead filter out of 16 is already 6.25%, and a later version should scale the floor to the layer's width.

The networks are small and were trained for 15 to 20 epochs on a laptop CPU. The transformer reaches only about 37% next character accuracy, and GPU training and large models were not tested. Finally, the third set of runs, with 60 healthy runs and the extra faults, was trained after the main results were in.

7 Conclusion

LHD compares four signals of every layer with ranges measured on healthy runs of the same network and stops a run when one signal stays out of range. On planted faults, it caught most broken runs within three epochs, usually long before the accuracy curve, and it rarely flagged healthy runs. Its clearest gain is on dead units, where a healthy range for each layer caught more faults than any fixed threshold while flagging few healthy runs, and the next step is to test it on real bugs from training code.

Use of generative AI

The author came up with the idea, designed the study in an earlier proposal, and wrote the core LHD library (signals, detector, and diagnosis), with Claude, made by Anthropic, providing function outlines and docstrings. Claude wrote the experiment scripts that trained and scored the networks, ran the training runs, wrote the analysis and figure code, and drafted parts of this paper, including the results and related work. The author directed the study, decided what to test and change, and is responsible for all of its content.

Code and data

The code, every saved run, and the tables behind each number are at https://github.com/sohan-bhat/lhd.

References

Amazon Web Services (n.d.a). List of Debugger built-in rules. Amazon SageMaker AI Developer Guide. https://docs.aws.amazon.com/sagemaker/latest/dg/debugger-built-in-rules.html

Amazon Web Services (n.d.b). Use Debugger built-in actions for rules. Amazon SageMaker AI Developer Guide. https://docs.aws.amazon.com/sagemaker/latest/dg/debugger-built-in-actions.html

Biewald, L. (2020). Experiment tracking with Weights and Biases. Software available from wandb.com. https://www.wandb.com/

Chwilczyński, S., Trębacz, K., Cyganik, K., Małecki, M., and Brzezinski, D. (2024). ART: Actually Robust Training. In *Machine Learning and Knowledge Discovery in Databases. Research Track and Demo Track (ECML PKDD 2024)*, Lecture Notes in Computer Science, vol. 14948, pp. 374-378. Springer, Cham. https://doi.org/10.1007/978-3-031-70371-3_23. Preprint: arXiv:2408.16285.

Groeger, F. (n.d.). wandb-callbacks: Additional callbacks for Weights & Biases. GitHub repository. https://github.com/FabianGroeger96/wandb-callbacks

Karpathy, A. (2019). A recipe for training neural networks. Blog post, April 25, 2019. https://karpathy.github.io/2019/04/25/recipe/

Kim, J., Humbatova, N., Jahangirova, G., Tonella, P., and Yoo, S. (2023). Repairing DNN architecture: Are we there yet? In *2023 IEEE Conference on Software Testing, Verification and Validation (ICST)*, pp. 234-245. IEEE. https://doi.org/10.1109/ICST57152.2023.00030. Preprint: arXiv:2301.11568.

Schoop, E., Huang, F., and Hartmann, B. (2021). UMLAUT: Debugging deep learning programs using program structure and model behavior. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*, pp. 1-16. ACM. https://doi.org/10.1145/3411764.3445538

Stallkamp, J., Schlipsing, M., Salmen, J., and Igel, C. (2012). Man vs. computer: Benchmarking machine learning algorithms for traffic sign recognition. *Neural Networks*, 32:323-332. https://doi.org/10.1016/j.neunet.2012.02.016

Wardat, M., Le, W., and Rajan, H. (2021). DeepLocalize: Fault localization for deep neural networks. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*, pp. 251-262. IEEE. https://doi.org/10.1109/ICSE43902.2021.00034

Wardat, M., Cruz, B. D., Le, W., and Rajan, H. (2022). DeepDiagnosis: Automatically diagnosing faults and recommending actionable fixes in deep learning programs. In *Proceedings of the 44th International Conference on Software Engineering (ICSE 2022)*, pp. 561-572. ACM, New York. https://doi.org/10.1145/3510003.3510071. Preprint: arXiv:2112.04036.

Zhang, X., Zhai, J., Ma, S., and Shen, C. (2021). AUTOTRAINER: An automatic DNN training problem detection and repair system. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*, pp. 359-371. IEEE. https://doi.org/10.1109/ICSE43902.2021.00043